

이펙티브 자바 아이템 10

`equals`는 일반 규약을 지켜 재정의하라

핵심 개념: equals의 본질

== 연산자	equals() 메서드
주소 비교 메모리상 완전히 같은 객체인가?	내용 비교 내부의 값이 논리적으로 같은가?

기본적으로 Object의 equals()는 ==와 동일하게 작동합니다.
따라서 논리적 내용 비교를 원한다면 반드시 재정의(Override)해야 합니다.

재정의를 피해야 하는 4가지 순간 (The "Do Not Touch" Rule)



① 본질적으로 고유한 인스턴스

값이 아닌 동작/기능을 나타내는 클래스
(예: Thread).



② 논리적 동치성 검사 불필요

설계자가 객체끼리 내용을 비교할 일이
없다고 판단한 경우.



③ 상위 클래스에서 완벽히 구현

부모 클래스가 구현한 **equals**가 하위
클래스에도 딱 들어맞는 경우.



④ 외부 호출이 금지된 경우

private 또는 package-private 클래스
이며 **equals**를 호출할 일이 없는 경우.
(실수 호출 방지용 예외 처리 필요)

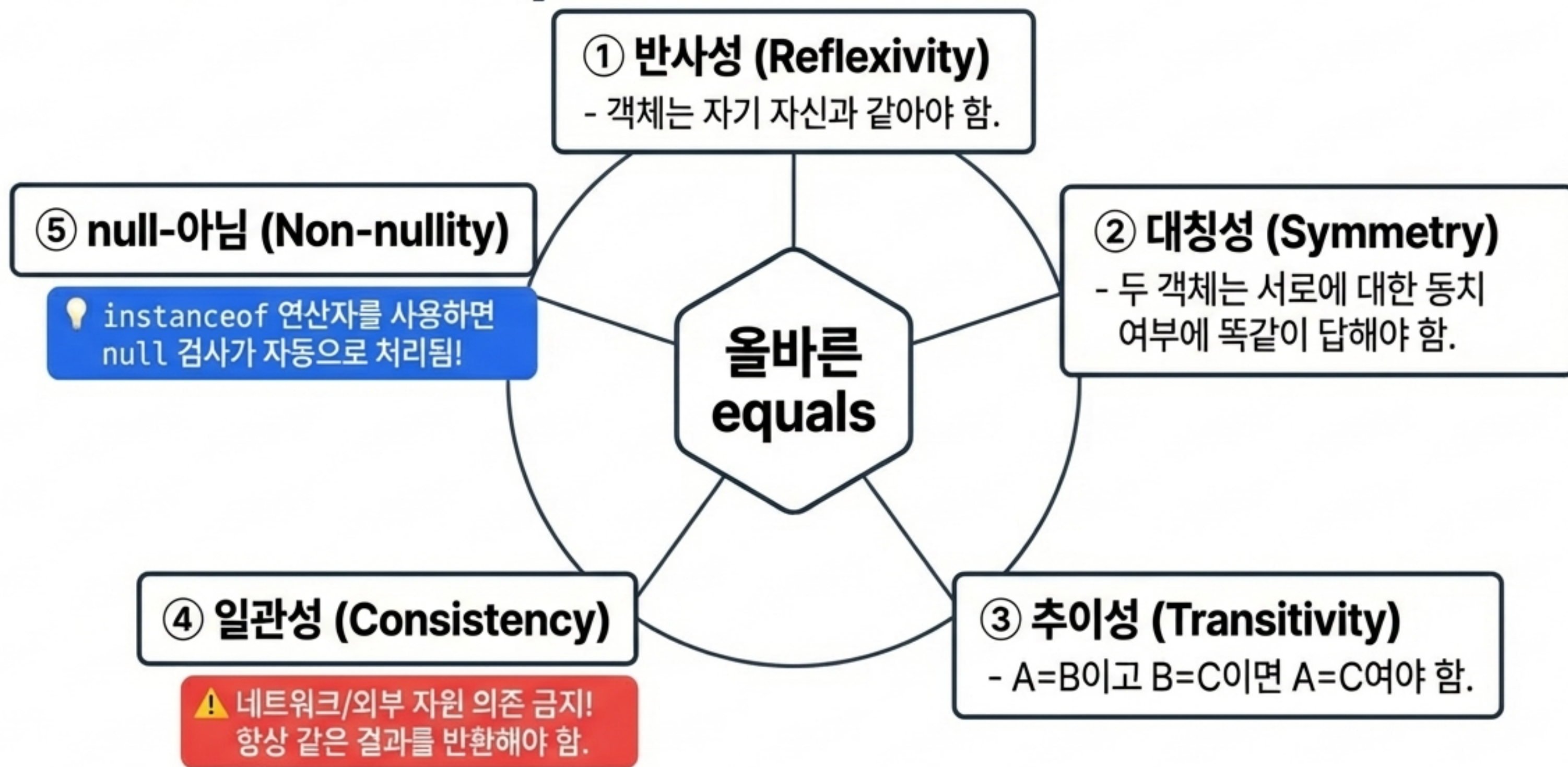
가장 중요한 단 하나의 원칙

A.equals(B)

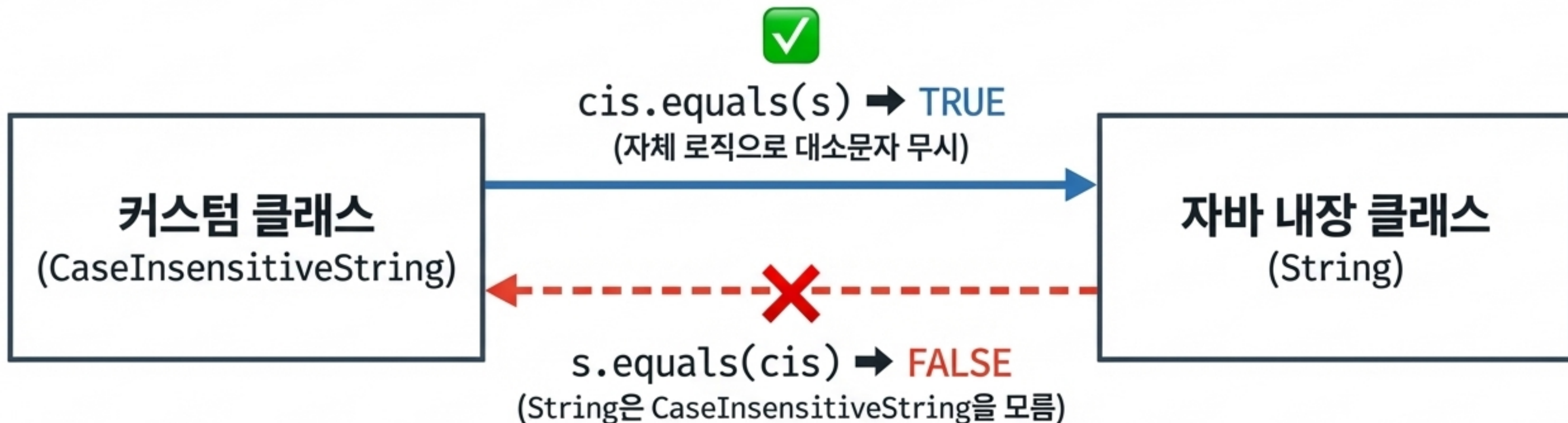
equals는 항상 주체(A)의 클래스에 정의된 코드가 실행됩니다!

이 비대칭적인 구조 때문에 A가 B를 평가하는 기준과 B가 A를 평가하는 기준이 달라질 수 있습니다. (다음 슬라이드들의 핵심 문제 원인)

절대 지켜야 할 5가지 규약 (General Contracts)



대칭성 위배의 뒤통수 (코드 10-1)

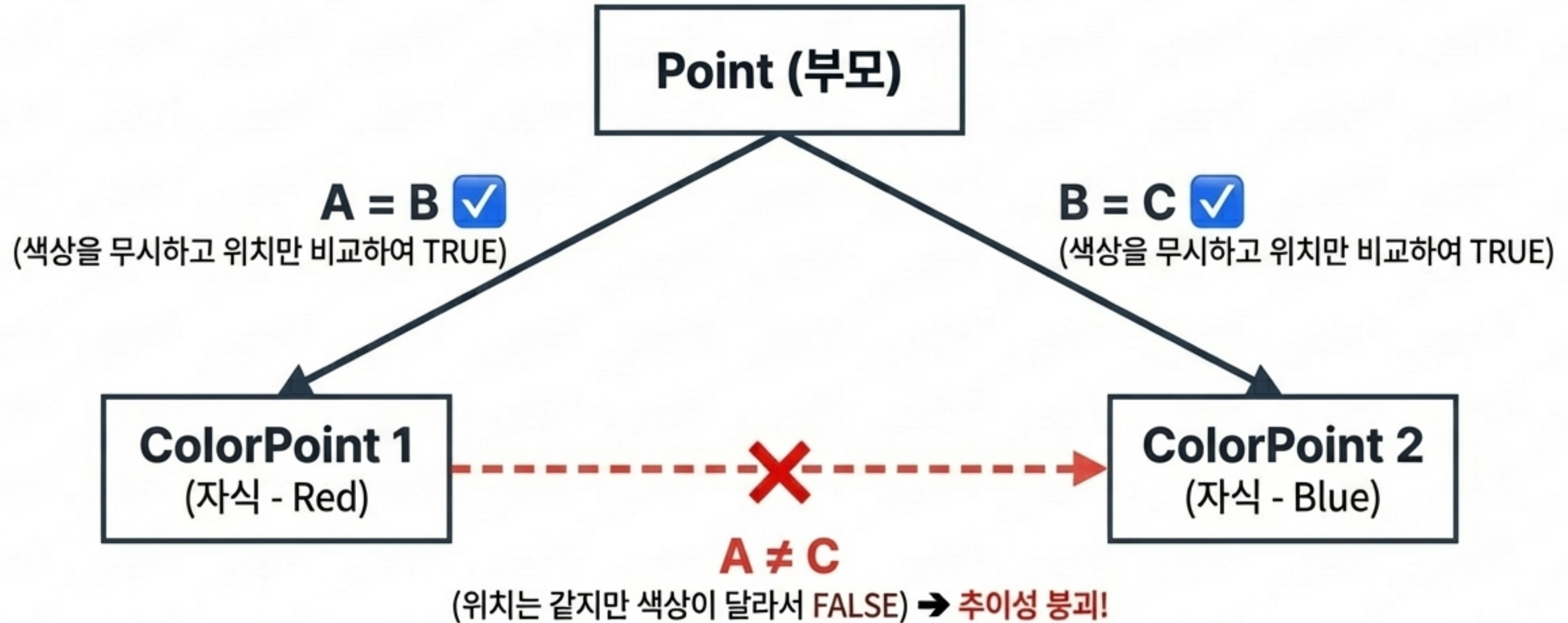


서로 다른 클래스다른 클래스 간에 억지로 `equals`를 연동하려 하면 대칭성이 붕괴됩니다.

핵심 분기점: instanceof vs getClass

instanceof	getClass
이 타입이거나 자식이면 참(true)	정확히 이 클래스여야만 참(true)
 👨👩 가족관계 (상속 관계 인정)	 🪪 주민등록증 대조 (상속 관계 불인정)
자식 클래스가 들어와도 true 반환.	정확히 같은 클래스일 때만 true 반환.
ColorPoint instanceof Point → TRUE	ColorPoint.getClass() == Point.class → FALSE

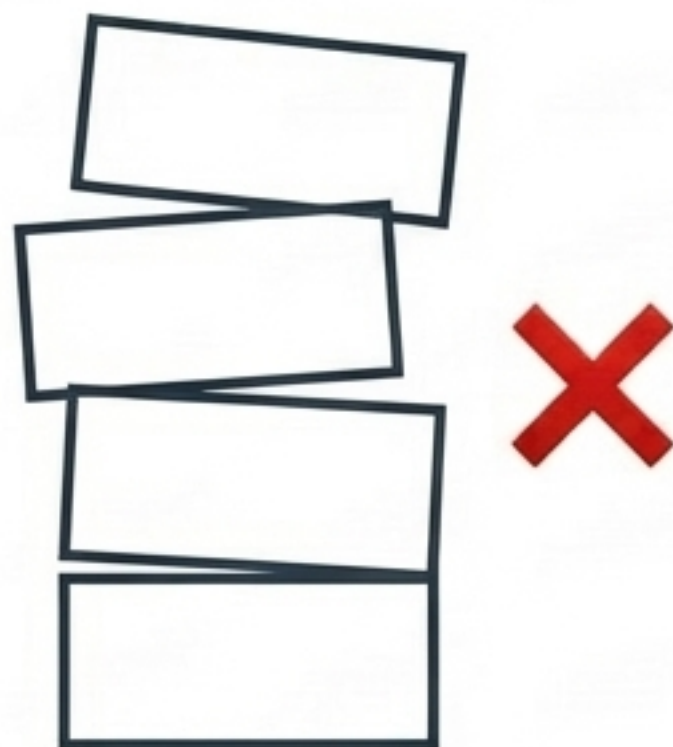
추이성 위배의 뒧 (상속의 한계)



구체 클래스를 확장(상속)해 새로운 값을 추가하면서 equals 규약을 만족시킬 방법은 존재하지 않습니다.

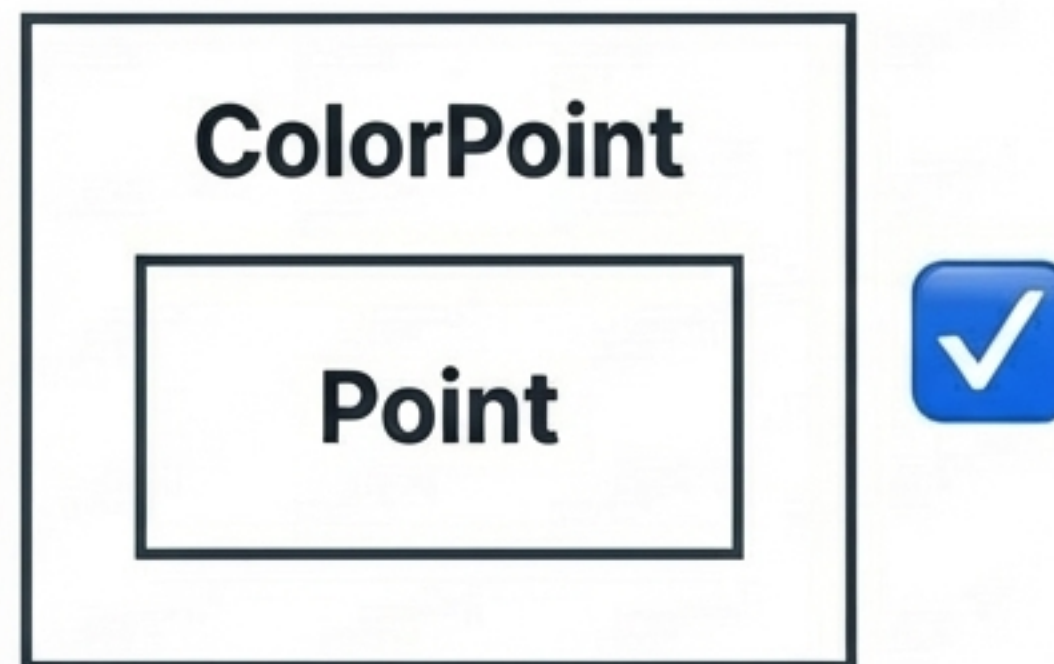
완벽한 해결책: 컴포지션 (Composition)

상속 (Inheritance) - ❌



수직적 확장으로 인해 부모/자식 간의
비교 로직이 엉키며 규약 파괴.

컴포지션 (Composition) - ✅



상속 대신 Point 객체를 ColorPoint의 private 필드로
선언! Point 뷰(view) 메서드를 반환하는 방식을
사용하여 구조적 충돌 원천 차단.

(코드 10-5) "우회해서 부품을 꺼내 비교하게 하라."

equals 완벽 구현 4단계 가이드

1

== 연산자로 자기 자신의 참조인지 확인

성능 최적화: 자기 자신이면 바로 true  반환.

2

instanceof 연산자로 입력이 올바른 타입인지 확인

잘못된 타입이 들어오면 false  반환 및 null 체크 동시 해결.

3

입력을 올바른 타입으로 형변환 (Casting)

2단계에서 instanceof를 통과했으므로 100% 안전함. 

4

입력 객체와 자기 자신의 대응되는 '핵심 필드'들이 모두 일치하는지 검사

float와 double은 특수 메서드로, 참조 타입은 equals로, 기본 타입은 ==로 비교.

모범 사례: PhoneNumber 클래스

```
@Override public boolean equals(Object o) {  
    if (o == this) return true;   
    if (!(o instanceof PhoneNumber)) return false;  
    PhoneNumber pn = (PhoneNumber) o;  
    return pn.lineNum == lineNum && pn.prefix == prefix &&  
           pn.areaCode == areaCode;  
}
```

①

②

③

④

모든 규약을 완벽히 준수하는 가장 전형적이고 깔끔한 형태.

치명적 실수 3가지 (Red Flag Warnings)



파라미터 타입은 반드시 Object!

`equals(MyClass o)`는 재정의가 아니라 다중정의(Overloading)입니다. 반드시 `@Override` 애너테이션을 달아 컴파일러의 검사를 받으세요.



`hashCode`도 반드시 함께 재정의!

`equals`를 재정의하고 `hashCode`를 그대로 두면 `HashMap`, `HashSet` 등에서 객체를 찾지 못하는 치명적 버그가 발생합니다. (아이템 11)



너무 복잡하게 만들지 말 것!

필드들의 동치성만 검사해도 충분합니다. 지나치게 복잡한 논리적 검사는 오히려 버그를 유발합니다.

"꼭 필요한 경우가 아니면, equals를 재정의하지 말자."

많은 경우 Object의 기본 equals가 정답입니다. 그럼에도 재정의해야 한다면,
상속 대신 컴포지션을 사용하고 5가지 규약과 핵심 필드 검사를 완벽하게 지켜야 합니다.